Sesion 17

Quantum K-means

Roadmap

0. Recap: why distances after Quantum Bayes
1. Classical K-means & where quantum helps
2. Encodings: amplitude vs angle (normalization)
3. Quantum similarity / distance estimators (swap, Hadamard, kernels)
4. From similarity to Euclidean distance (formulas)
5. Hybrid K-means loops (variant A: direct distances; variant B: quantum kernel K-means)
6. Qiskit implementation patterns (primitives: `Sampler`, `Estimator`) — runnable helpers
7. Performance & scalability accounting (shots, state-prep, N, K)
8. Evaluation protocol & metrics
9. Practical tips & pitfalls checklist
10. Mini-exercises + worked solutions (appendix)

0) Recap — from Quantum Bayes to K-means

Quantum Bayes focused on *probability/expectation* queries; clustering needs *distances/inner products* between vectors. Circuits that estimate overlaps or kernel values can be slotted into the assignment step of Lloyd's algorithm, potentially offering improved sample complexity for similarity estimation in ideal settings. On NISQ, hybrid approaches and kernel variants are the most practical.

1) Classical K-means (Lloyd's) — where to place quantum work

Lloyd's loop:

1. Assignment: for each point x_i, find cluster index $c = \arg\min_k \|x_i - \mu_k\|^2$.
2. Update: $\mu_k \leftarrow \frac{1}{|C_k|} \sum_{x \in C_k} x$.

   Computational hot spot: computing $\|x_i - \mu_k\|^2$ for many (i, k). Replace or augment these computations with quantum similarity estimates.

2) Encodings — two practical options

Amplitude encoding

- Map $x \in \mathbb{R}^d$ to $|x\rangle = \sum_j x_j / \|x\| \, |j\rangle$ on $n = \lceil \log_2 d \rceil$ qubits.
- **Pros:** compact (log qubits).
- **Cons:** state preparation depth can be large; fragile to noise. Use for simulation/controlled experiments or when fast state-prep oracle exists.

Angle (feature) encoding (recommended for NISQ)

- Map each feature to rotation(s): e.g. $R_y(\alpha x_j)$ on qubit j, possibly with entanglers (ZZ feature map).
- **Pros:** shallow, hardware-friendly.
- **Cons:** uses ≈d qubits; induces an implicit kernel rather than exact amplitude overlap.

Normalization rules

- Amplitude encoding: L2 normalize vectors before state prep.
- Angle encoding: standardize features (zero mean, unit variance) then scale to the angle domain $[-\pi, \pi]$ or $[-\pi/2, \pi/2]$ as appropriate.

3) Quantum distance & similarity estimation

3.1 Swap test (overlap for amplitude encoding)

If you can prepare $|x\rangle$ and $|y\rangle$ on registers, the swap test estimates $|\langle x|y\rangle|^2$:

$$\Pr(\text{ancilla} = 0) = \tfrac{1}{2}\big(1 + |\langle x|y\rangle|^2\big).$$

From $|\langle x|y\rangle|$ and L2-normalization you can get cosine similarity or Euclidean distance.

Circuit pattern

- Ancilla H → repeated CSWAPs between register qubits → ancilla H → measure ancilla.

Shot behavior

- Variance of ancilla outcome $p(1-p)$. Shots S give std ≈ $\sqrt{p(1-p)/S}$. Use ~512–2048 shots for stable estimates in small experiments.

3.2 Hadamard test (real/imag inner product)

- Estimates $\mathrm{Re}\langle x|U|x\rangle$ using ancilla H, controlled-U, ancilla H; repeat with phase shifts for imaginary part. If you can build U such that $U|x\rangle = |y\rangle$ (or phase-encode y), the Hadamard test yields signed inner products.

Practicality

- More flexible but requires constructing controlled-U; better when you can build U cheaply (e.g., U is a simple feature map difference).

3.3 Quantum kernel with angle/ZZ feature map

For feature map $U_\phi(x)$,

$$K(x, y) = |\langle 0|U_\phi^\dagger(x)U_\phi(y)|0\rangle|^2.$$

Estimate $K(x, y)$ by preparing $U_\phi(y)U_\phi^\dagger(x)|0\rangle$ and measuring probability of $|0\rangle$. This is shallow for many feature maps and preferred for NISQ.

3.4 Precision & amplitude estimation

- Plain sampling: std error $\propto 1/\sqrt{S}$.
- Amplitude estimation (AE) can reduce oracle calls from $O(1/\epsilon^2)$ to $O(1/\epsilon)$ asymptotically, but AE requires deeper circuits (Grover operators) and AE wrappers may be unstable on 2.x; iterative/ML variants exist — implement on a per-case basis.

4) From similarity → Euclidean distance

For L2-normalized vectors $\|x\| = \|y\| = 1$:

- Inner product $s = \langle x|y\rangle$ (real) → Euclidean:

$$\|x - y\|^2 = 2(1 - s).$$

- Swap test gives $|s|^2$; convert accordingly:
  - If sign matters (s could be negative), use Hadamard test to get signed s.
  - If only magnitude available, note: $\|x - y\|^2 = 2(1 \pm \sqrt{|s|^2})$ — ambiguous sign. Use signed method when sign is important.

For kernel methods: derive cluster assignment from kernel distances:

$$\|\phi(x) - m_k\|^2 = K(x,x) - \tfrac{2}{n_k}\sum_{i\in C_k} K(x,x_i) + \tfrac{1}{n_k^2}\sum_{i,j\in C_k} K(x_i,x_j).$$

5) Hybrid K-means variants

Variant A — Quantum distances (swap/Hadamard)

1. Initialize centroids $\{\mu_k\}$ classically.
2. For each point x_i and centroid μ_k compute similarity s_{ik} (swap or Hadamard).
3. Assign $x_i \to \arg\min_k 2(1 - s_{ik})$ (convert similarity → distance).
4. Update μ_k classically (mean of assigned points).
5. Repeat until convergence.

Notes

- Use angle encoding if amplitude preparation is expensive.
- For Hadamard tests, implement controlled U carefully for sign.

Variant B — Quantum kernel K-means (recommended for NISQ)

1. Choose shallow feature map U_ϕ (ZZFeatureMap or simple RY layers).
2. Compute kernel matrix $K_{ij} = K(x_i, x_j)$ via `Sampler` / `Estimator`, cache symmetric entries.
3. Run kernel K-means (spectral trick or precomputed kernel options in libraries).
4. Update clusters classically.

Why B is practical

- No centroids stored in quantum memory.
- Shallow circuits for kernel entries; cheap to reuse and batch.
- Easier to compare to classical kernel baselines.

6) Qiskit implementations — primitives-first (modern, 2.x-friendly)

Below are small, tested helper functions you can plug into experiments. They use `qiskit.primitives.Sampler` / `Estimator`. (If you use `Sampler` on hardware, prefer batching and Runtime to reduce latency.)

> **Install**: `pip install qiskit qiskit-aer numpy scikit-learn`

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.manifold import SpectralEmbedding
from sklearn.metrics import adjusted_rand_score
from qiskit.circuit.library import zz_feature_map
from qiskit import QuantumCircuit
from qiskit_aer.primitives import SamplerV2 as Sampler
```

```
# ---------- Parameters ----------
N_POINTS = 8        # very small for classroom demo
SHOTS = 2048         # small shot budget; increase for stability
REPS = 3            # feature map repetitions
RANDOM_SEED = 0

# ---------- 1) Small toy dataset ----------
X_raw, y_true = make_moons(n_samples=N_POINTS, noise=0.12, random_state=RANDOM_SEED)
scaler = StandardScaler().fit(X_raw)
X_scaled = scaler.transform(X_raw)          # standardize for angle mapping
```

```python
# map features into angle domain [-pi, pi] simply (small demo)
X_angles = (X_scaled / np.max(np.abs(X_scaled))) * np.pi

# ---------- 2) Define a shallow feature map ----------
feature_map = zz_feature_map(feature_dimension=2, reps=REPS)  # 2D input expected
# feature_map.parameters order typically [x0, x1]; ensure we supply matching arrays
```

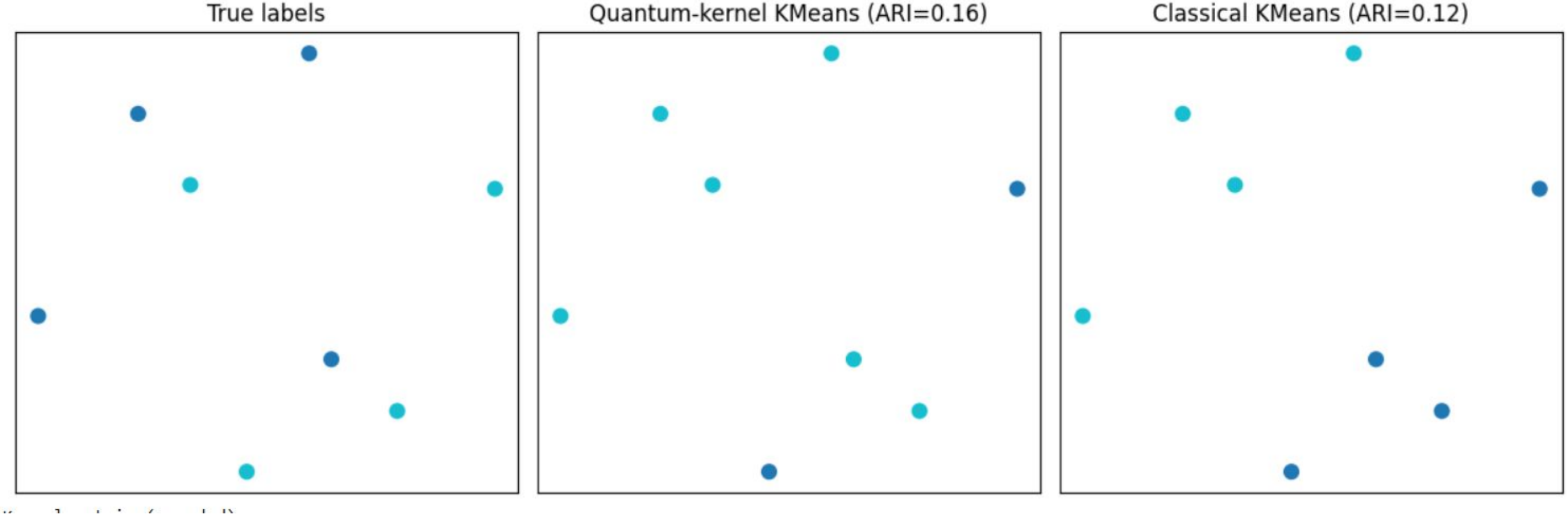
```python
def compute_kernel_entry_simple(x, y, fmap, shots=512):
    """Bind x,y into a circuit U(y) * U(x)^dagger and measure probability of |0..
    params = list(fmap.parameters)
    if len(params) != len(x):
        raise ValueError("Input length doesn't match feature map parameters")
    Ux = fmap.assign_parameters({p: float(v) for p, v in zip(params, x)})
    Uy = fmap.assign_parameters({p: float(v) for p, v in zip(params, y)})
    qc = QuantumCircuit(fmap.num_qubits, fmap.num_qubits)
    qc.compose(Uy, inplace=True)
    qc.compose(Ux.inverse(), inplace=True)
    qc.measure_all()
    sampler = Sampler()
    job = sampler.run([qc], shots=shots)
    res = job.result()
    pdict =res[0].data.meas.get_counts()
    key = '0' * fmap.num_qubits
    p0 = pdict.get(key, 0.0)
    return float(p0)
```

```
N = len(X_angles)
Kmat = np.eye(N)
print("Computing small quantum kernel matrix ({} points, {} shots each entry)...".format(N, SHOTS))
for i in range(N):
    for j in range(i+1, N):
        k = compute_kernel_entry_simple(X_angles[i], X_angles[j], feature_map, shots=SHOTS)
        Kmat[i, j] = Kmat[j, i] = k

print("Kernel matrix computed (shape {})".format(Kmat.shape))
```

```python
# ---------- 5) Kernel K-means via spectral embedding -> KMeans ----------
n_clusters = 2
Z = SpectralEmbedding(n_components=n_clusters, affinity='precomputed').fit_transform(Kmat)
labels_qk = KMeans(n_clusters=n_clusters, n_init=10, random_state=RANDOM_SEED).fit_predict(Z)

# Baseline classical KMeans on Euclidean scaled inputs
labels_km = KMeans(n_clusters=n_clusters, n_init=10, random_state=RANDOM_SEED).fit_predict(X_scaled)

# ---------- 6) Evaluate and plot ----------
ari_qk = adjusted_rand_score(y_true, labels_qk)
ari_km = adjusted_rand_score(y_true, labels_km)
print(f"Adjusted Rand Index (Quantum-kernel KMeans): {ari_qk:.3f}")
print(f"Adjusted Rand Index (Classical KMeans):       {ari_km:.3f}")
```

```python
# plot results: raw data with true labels, quantum-kernel labels, classical kmeans labels
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].scatter(X_raw[:,0], X_raw[:,1], c=y_true, cmap='tab10', s=60)
axes[0].set_title("True labels")
axes[1].scatter(X_raw[:,0], X_raw[:,1], c=labels_qk, cmap='tab10', s=60)
axes[1].set_title(f"Quantum-kernel KMeans (ARI={ari_qk:.2f})")
axes[2].scatter(X_raw[:,0], X_raw[:,1], c=labels_km, cmap='tab10', s=60)
axes[2].set_title(f"Classical KMeans (ARI={ari_km:.2f})")
for ax in axes:
    ax.set_xticks([]); ax.set_yticks([])
plt.tight_layout()
plt.show()

# ---------- 7) Quick diagnostics: print kernel matrix ----------
print("Kernel matrix (rounded):")
with np.printoptions(precision=3, suppress=True):
    print(Kmat)
```

```
Computing small quantum kernel matrix (8 points, 2048 shots each entry)...
Kernel matrix computed (shape (8, 8))
Adjusted Rand Index (Quantum-kernel KMeans): 0.160
Adjusted Rand Index (Classical KMeans):      0.125
```



7) Performance & scalability (cost accounting)

Variables: N points, K clusters, d dimension, S shots, P parameters in AE (if used).

Variant A — swap/Hadamard distances:

- For each assignment iteration you need $N \times K$ similarity estimates.
- Each estimate may cost S shots and state-prep cost T_{prep}. So per iteration: cost ≈ $NK(T_{\text{prep}} + S \cdot T_{\text{meas}})$. State prep often dominates for amplitude encoding.

Variant B — kernel K-means:

- Precompute full $N \times N$ kernel: $O(N^2)$ kernel entries (or $N(N-1)/2$ with symmetry). Good for small N (lab scale).
- Each kernel entry costs S shots and shallow circuit time for angle maps.

When advantage could appear:

- When d is huge and amplitude encoding uses $\log d$ qubits with an efficient stateprep oracle, and assignment costs classical $O(NKd)$ per iteration so quantum can reduce the per pair cost. In practice, state-prep cost and noise are the limiting factors.

8) Evaluation protocol & metrics

Metrics

- Internal (unsupervised): Silhouette score, Davies–Bouldin index.
- External (if labels): Adjusted Rand Index (ARI), Normalized Mutual Information (NMI).

Protocol

1. Standardize features. For angle maps scale to $[-\pi, \pi]$. For amplitude encoding L2-normalize each vector.
2. Fix a shot budget (e.g., 256,512,1024,2048) and random seeds. Report mean±std over seeds.
3. Compare to baselines: (a) classical K-means, (b) kernel K-means (RBF) with tuned hyperparams, (c) spectral clustering.
4. Ablation axes: feature map depth (reps), shots, noise model, state-prep method.

Reporting

- Accuracy metrics + shot/latency cost (approx wall-time if available).
- For kernel methods report matrix compute time and memory.

9) Practical tips & pitfalls (quick checklist)

- **State prep is the hidden cost.** For amplitude encoding prefer small d or a fast oracle; otherwise use angle maps.
- **Normalization matters.** L2 normalize for amplitude overlap; standardize then scale for angle maps.
- **Sign ambiguity with swap test:** swap gives $|\langle x|y\rangle|^2$ — if sign matters use Hadamard test.
- **Shot budgeting:** start with low shots for early iterations (256–512), increase near convergence.
- **Batching & caching:** batch many kernel evaluations in single `Sampler` calls; cache symmetric kernel entries.
- **Topology & transpile:** set optimization level 2–3; bind parameters to avoid repeated transpilation.
- **Mitigation:** readout calibration improves kernel reliability; ZNE useful for small kernel corrections.
- **Fair baselines:** match parameter count and depth for classical baseline kernels.